

Book Store Requirements, Architecture, and Patterns

Dakota Wynn

Lucas Bramer

Jason Reid

Software Design and Architecture

University of Central Missouri – UCM

November 10, 2025

Project Scope:

User will be able to browse a books catalog.

User will be able to register and login.

User will be able to manage an individual cart before placing an order.

User will be able to simulate payment and ordering.

Admin will be able to add/update books.

Stretch: User will be notified when order is placed.

Architecture + Requirements: Three-Tiered

Frontend: Vue, JS, HTML/CSS, Node

Backend: Django, Python

Database: MySQL, AWS RDS

Entity Relationship Diagram:

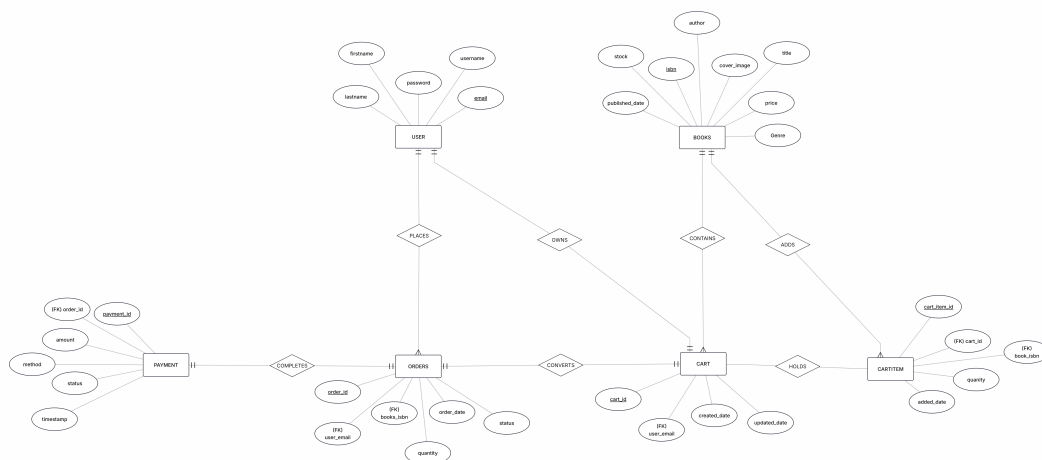


Figure 1: Entity Relationship Diagram

ERD and Database Explained:

User -owns→ Cart -contains→ CartItem -adds→ Books

Cart -converts→ Order -complete→ Payment

User -places→ Order

Patterns Definitions:

Singleton: Ensures a class has only one instance, and provides a global point of access to it.

Strategy: Defines a family of algorithms, encapsulates each one, and makes them interchangeable.

Factory: Defines an interface for creating an object, but lets sub-classes decide which class to instantiate. Factory method lets a class defer instantiation to sub-classes.

Observer: Defines a one-to-many dependency between objects so that when one object changes state, all its dependents are notified and updated automatically.

State: Allow an object to alter behavior when its internal state changes. It will appear to change its class.

Unified Modeling Language (UMLs):

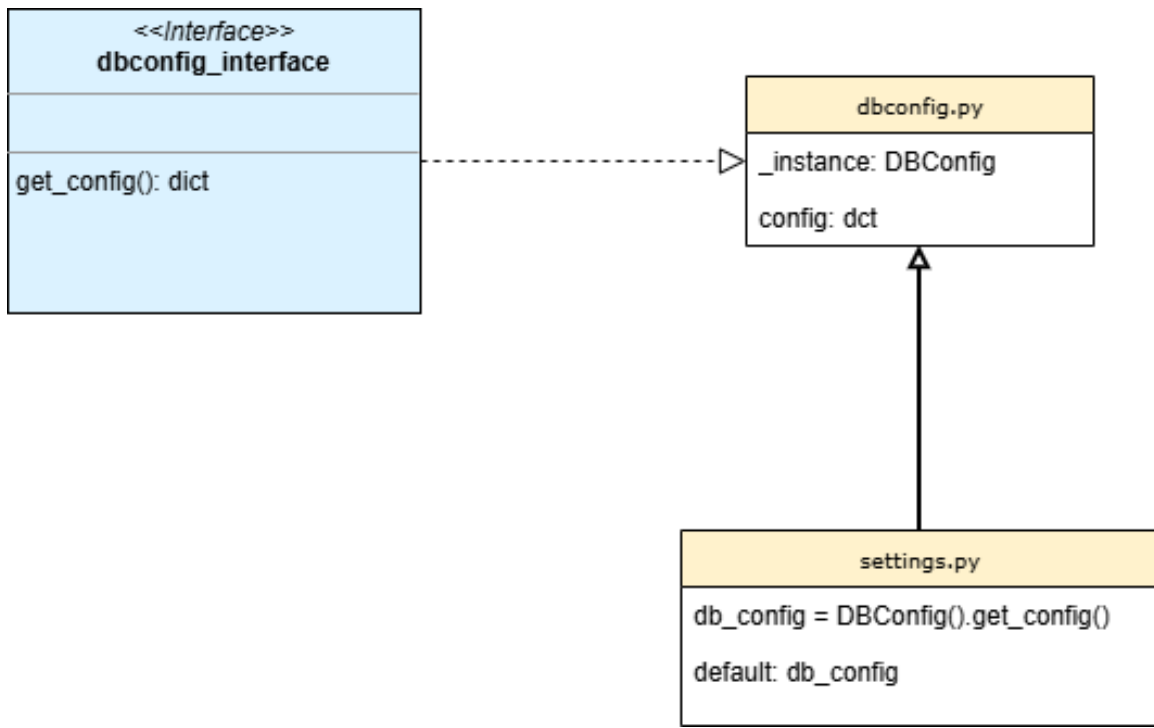


Figure 2: Singleton UML Diagram

UMLs Explained:

Singleton:

Strategy:

Factory:

Observer:

State:

Challenges:

Research:
Database:
Backend:
Frontend:

Future Implementations:

Updated Catalog:
Text Updates:
Payment System Actualized:

Research: